Supervised Machine Learning Models

Week 2 Task Report - Regression and Classification with Scikit-learn

Task	Train at least two machine learning models and compare their performance using evaluation metrics
Prepared by	Shubh
Date	28 October 2026
Tools	Python 3, Scikit-learn, Pandas, NumPy, Matplotlib
Data	Cleaned loan applicant dataset from Week 1 (515 rows, 15 columns)
Models	Linear Regression, Logistic Regression, Decision Tree, Random Forest, K-Nearest Neighbors (5 algorithms, 8 trained models)
Code	src/train_models.py (full code and results in the GitHub repository)

Contents

- 1. Objective and overview
- 2. The five algorithms
- 3. How models are evaluated
- 4. Step-by-step documentation (Steps 1 to 6)
- 5. Final model comparison
- 6. Conclusion
- 7. Limitations and next steps
- Appendix: repository and how to run

1. Objective and overview

In supervised learning a model learns from examples where the correct answer is known, then predicts the answer for new data. This report covers the Week 2 objectives:

- Implement Linear Regression, Logistic Regression, Decision Tree, Random Forest and K-Nearest Neighbors with Scikit-learn.
- Learn model training, prediction and evaluation with performance metrics.
- **Deliverable:** train at least two models and compare their performance. Here, **eight models** are trained and compared.

Two problems are solved with the cleaned dataset from Week 1: a **regression** problem (predict a loan applicant's annual income) and a **classification** problem (predict whether the loan is approved). Regression uses Linear Regression, KNN, Decision Tree and Random Forest. Classification uses Logistic Regression, KNN, Decision Tree and Random Forest.

2. The five algorithms

Algorithm	Used for	How it works	Strengths	Weaknesses
Linear Regression	Regression	Fits a weighted sum of the features that best matches the target number.	Simple, fast, coefficients are easy to explain.	Cannot capture curved or complex patterns.
Logistic Regression	Classification	Like linear regression, but an S-shaped function turns the result into a probability between 0 and 1.	Simple, gives probabilities, hard to overfit.	Only draws a straight boundary between classes.
Decision Tree	Both	Asks a chain of yes/no questions about the features to reach a prediction.	Easy to read, handles non-linear patterns.	Overfits easily; small data changes alter the tree.
Random Forest	Both	Trains many trees on random parts of the data and averages their predictions.	Accurate, robust, shows feature importance.	Slower and harder to explain.
K-Nearest Neighbors	Both	Predicts from the k most similar training examples.	No training step, easy idea.	Needs scaled features; sensitive to k and to irrelevant columns.

3. How models are evaluated

Metric	Problem	Meaning
MAE	Regression	Average size of the prediction error (lower is better).
RMSE	Regression	Like MAE but punishes large errors more (lower is better).
R2	Regression	Share of the variation in the target that the model explains: 1 is perfect, 0 is no better than always predicting the average.
Accuracy	Classification	Share of all predictions that are correct.
Precision	Classification	Of the applicants predicted as approved, how many really were.
Recall	Classification	Of the applicants who really were approved, how many the model found.
F1 score	Classification	Balance of precision and recall in one number.
ROC-AUC	Classification	How well the model ranks approved above rejected applicants: 0.5 is guessing, 1.0 is perfect.
Confusion matrix	Classification	Table of correct and wrong predictions for each class.

Metric	Problem	Meaning
Cross-validation	Both	Trains and tests on 5 different splits of the training data and averages the score, so one lucky split does not decide.

4. Step-by-step documentation

Each step shows what is done and why, the exact code that was run, its output, and the result. The code comes straight from **src/train_models.py**.

Step 1: Load the data and define the two prediction problems

Both tasks use the cleaned dataset from Week 1 (515 rows, every column numeric). Two prediction problems are defined. **Regression** predicts a number: Annual_Income. **Classification** predicts a category: Approved (1 = approved, 0 = rejected). The target column is removed from the features (X) so the model cannot see the answer. The features are already standardised (z-scores), so regression errors are in z-score units and are converted back to rupees with the income standard deviation from Week 1 (Rs. 227,053.6).

CODE

```
df = pd.read_csv("data/loan_applicants_clean.csv")
print("Shape:", df.shape)
print("All columns numeric:", all(pd.api.types.is_numeric_dtype(t) for t in df.dtypes))

# Regression problem: predict Annual_Income (a continuous number, stored as a z-score)
X_reg, y_reg = df.drop(columns=["Annual_Income", "Approved"]), df["Annual_Income"]
# Classification problem: predict Approved (0 = rejected, 1 = approved)
X_clf, y_clf = df.drop(columns=["Approved"]), df["Approved"]
# Standard deviation of Annual_Income before scaling (Week 1, Step 11) - converts errors back to Rs.
INCOME_STD = 227053.6

print("\nRegression features :", list(X_reg.columns))
print("Regression target   : Annual_Income")
print("\nClassification target balance (%):")
print((y_clf.value_counts(normalize=True) * 100).round(1))
```

OUTPUT

```
Shape: (515, 15)
All columns numeric: True

Regression features : ['Age', 'Gender', 'Education', 'Credit_Score', 'Loan_Amount', 'Loan_Term_Months', 'City_Delhi',
    'City_Kanpur', 'City_Lucknow', 'City_Mumbai', 'Employment_Status_Salaried', 'Employment_Status_Self-Employed',
    'Employment_Status_Unemployed']
Regression target   : Annual_Income

Classification target balance (%):
Approved
1    61.4
0    38.6
Name: proportion, dtype: float64
```

Result: 13 features for regression and 14 for classification. 61.4% of applicants are approved, so the two classes are reasonably balanced.

Step 2: Split into training and test sets

A model must be tested on data it has never seen, otherwise the score only measures memorisation. The data is split 80% / 20% with **train_test_split**. **random_state=42** makes the split repeatable and **stratify** keeps the same approval rate in both parts of the classification split.

CODE

```
Xr_train, Xr_test, yr_train, yr_test = train_test_split(X_reg, y_reg, test_size=0.2, random_state=42)
Xc_train, Xc_test, yc_train, yc_test = train_test_split(
    X_clf, y_clf, test_size=0.2, random_state=42, stratify=y_clf)   # stratify keeps the class balance
print("Regression     - train:", Xr_train.shape, " test:", Xr_test.shape)
print("Classification - train:", Xc_train.shape, " test:", Xc_test.shape)
print("Approval rate in train / test (%):",
      round(yc_train.mean() * 100, 1), "/", round(yc_test.mean() * 100, 1))
```

OUTPUT

```
Regression     - train: (412, 13)  test: (103, 13)
Classification - train: (412, 14)  test: (103, 14)
Approval rate in train / test (%): 61.4 / 61.2
```

Result: 412 rows for training and 103 rows for testing. The approval rate is 61.4% in the training set and 61.2% in the test set.

Step 3: Regression models - Linear Regression, KNN, Decision Tree, Random Forest

Four regression models are trained with **fit()**, used to **predict()** on the test set, and scored with MAE, RMSE and R2 (explained in Section 3). **5-fold cross-validation** on the training data (CV R2) shows how stable each score is across different slices of the data.

CODE

```
reg_models = {
    "Linear Regression": LinearRegression(),
    "KNN Regressor": KNeighborsRegressor(n_neighbors=7),
    "Decision Tree": DecisionTreeRegressor(max_depth=5, random_state=42),
    "Random Forest": RandomForestRegressor(n_estimators=200, max_depth=8, random_state=42),
}
kf = KFold(n_splits=5, shuffle=True, random_state=42)
rows, reg_pred = [], {}
for name, model in reg_models.items():
    model.fit(Xr_train, yr_train)                       # train
    pred = model.predict(Xr_test)                       # predict on unseen data
    reg_pred[name] = pred
    mae = mean_absolute_error(yr_test, pred)
    rows.append({"Model": name, "MAE": mae,
                 "RMSE": np.sqrt(mean_squared_error(yr_test, pred)),
                 "R2": r2_score(yr_test, pred),
                 "CV_R2": cross_val_score(model, Xr_train, yr_train, cv=kf, scoring="r2").mean(),
                 "MAE_Rs": mae * INCOME_STD})
reg_results = pd.DataFrame(rows).set_index("Model")
print(reg_results.sort_values("R2", ascending=False).round({"MAE": 3, "RMSE": 3, "R2": 3, "CV_R2": 3, "MAE_Rs": 0}))
print("\nLinear Regression coefficients (effect of each feature on income, in z-score units):")
print(pd.Series(reg_models["Linear Regression"].coef_, index=X_reg.columns).sort_values().round(3))
best_reg = reg_results["CV_R2"].idxmax()        # chosen by cross-validation, not by the test set
plot_regression(reg_results, yr_test, reg_pred[best_reg], best_reg)
reg_results.to_csv(f"{OUT}/regression_results.csv")
```

OUTPUT

```
                    MAE   RMSE     R2  CV_R2    MAE_Rs
Model
Random Forest     0.312  0.441  0.791  0.688   70812.0
Linear Regression 0.328  0.473  0.760  0.701   74400.0
Decision Tree     0.377  0.504  0.727  0.547   85509.0
KNN Regressor     0.463  0.568  0.654  0.598  105032.0

Linear Regression coefficients (effect of each feature on income, in z-score units):
Employment_Status_Self-Employed   -0.071
Loan_Term_Months                  -0.068
Credit_Score                      -0.009
City_Kanpur                        0.025
City_Mumbai                        0.032
Employment_Status_Salaried         0.035
Gender                             0.041
Employment_Status_Unemployed       0.067
Loan_Amount                        0.070
City_Delhi                         0.095
City_Lucknow                       0.133
Education                          0.517
Age                                0.668
dtype: float64
```

Regression results (sorted by cross-validated R2).

Model	MAE	RMSE	R2 (test)	R2 (cross-val.)	Average error in Rs.
Linear Regression	0.328	0.473	0.760	0.701	74,400
Random Forest	0.312	0.441	0.791	0.688	70,812
KNN Regressor	0.463	0.568	0.654	0.598	105,032
Decision Tree	0.377	0.504	0.727	0.547	85,509

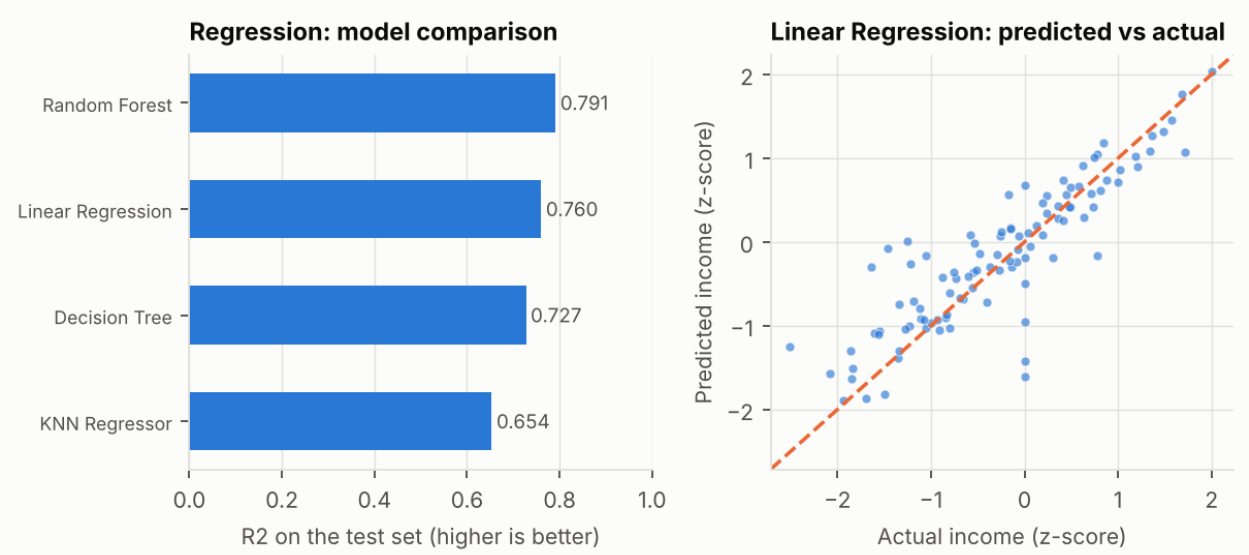


Figure 1 - Left: test R2 of the four regression models. Right: predicted vs actual income for the chosen model; points on the dashed line are perfect predictions.

Result: Random Forest has the best test R2 (0.791), but Linear Regression has the best cross-validated R2 (0.701 vs 0.688), so the two are practically tied. **Linear Regression is chosen** because cross-validation is a more reliable guide than one 103-row test set, and the model is simpler. Its average error is about Rs. 74,400. Age (0.67) and Education (0.52) are the main drivers of income. KNN is the weakest (R2 0.654), probably because distances work poorly with many 0/1 dummy columns.

Step 4: Classification models - Logistic Regression, KNN, Decision Tree, Random Forest

The four classification models are trained the same way and scored with accuracy, precision, recall, F1 and ROC-AUC, plus cross-validated accuracy. These first results use sensible default settings.

CODE

```
clf_models = {
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "KNN": KNeighborsClassifier(n_neighbors=7),
    "Decision Tree": DecisionTreeClassifier(max_depth=5, random_state=42),
    "Random Forest": RandomForestClassifier(n_estimators=200, max_depth=8, random_state=42),
}
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
rows = []
for name, model in clf_models.items():
    model.fit(Xc_train, yc_train)
    pred = model.predict(Xc_test)
    proba = model.predict_proba(Xc_test)[:, 1]          # probability of "approved"
    rows.append({"Model": name,
                 "Accuracy": accuracy_score(yc_test, pred),
                 "Precision": precision_score(yc_test, pred),
                 "Recall": recall_score(yc_test, pred),
                 "F1": f1_score(yc_test, pred),
                 "ROC_AUC": roc_auc_score(yc_test, proba),
                 "CV_Accuracy": cross_val_score(model, Xc_train, yc_train, cv=skf, scoring="accuracy").mean()})
clf_results = pd.DataFrame(rows).set_index("Model")
print(clf_results.sort_values("ROC_AUC", ascending=False).round(3))
plot_metrics(clf_results, "02_classification_metrics", "Classification: default models on the test set")
clf_results.to_csv(f"{OUT}/classification_results_default.csv")
```

OUTPUT

```
                     Accuracy  Precision  Recall     F1  ROC_AUC  CV_Accuracy
```

```
Model
Logistic Regression     0.728      0.746   0.841  0.791    0.758        0.697
Random Forest           0.738      0.750   0.857  0.800    0.748        0.660
KNN                     0.709      0.714   0.873  0.786    0.730        0.670
Decision Tree           0.660      0.719   0.730  0.724    0.643        0.650
```

Classification results with default settings (sorted by ROC-AUC).

Model	Accuracy	Precision	Recall	F1	ROC-AUC	CV accuracy
Logistic Regression	0.728	0.746	0.841	0.791	0.758	0.697
Random Forest	0.738	0.750	0.857	0.800	0.748	0.660
KNN	0.709	0.714	0.873	0.786	0.730	0.670
Decision Tree	0.660	0.719	0.730	0.724	0.643	0.650

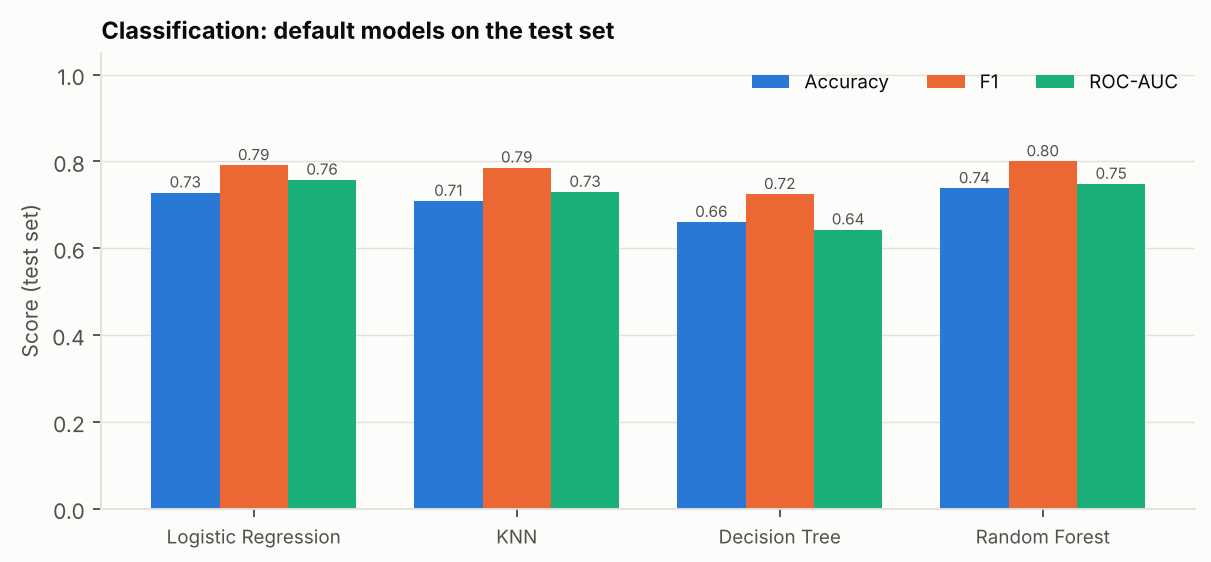


Figure 2 - Accuracy, F1 and ROC-AUC of the four default classifiers on the test set.

Result: All four models beat random guessing. Random Forest has the highest accuracy (0.738) and F1 (0.800); Logistic Regression has the highest ROC-AUC (0.758). The untuned Decision Tree is the weakest (accuracy 0.660, AUC 0.643) because a single deep tree is unstable. The top three models are close: with only 103 test rows, one applicant equals about one percentage point.

Step 5: Hyperparameter tuning with GridSearchCV

Default settings are only a first guess. **GridSearchCV** tries every combination of chosen hyperparameters with 5-fold cross-validation on the **training data only** (the test set stays untouched) and keeps the combination with the best ROC-AUC. Tuned settings: C (regularisation strength) for Logistic Regression, k (number of neighbours) for KNN, depth and leaf size for the tree, and number of trees, depth and leaf size for the forest.

CODE

```
grids = {
    "Logistic Regression": (LogisticRegression(max_iter=1000), {"C": [0.01, 0.1, 1, 10]}),
    "KNN": (KNeighborsClassifier(), {"n_neighbors": [3, 5, 7, 11, 15, 21]}),
    "Decision Tree": (DecisionTreeClassifier(random_state=42),
                      {"max_depth": [2, 3, 4, 5, 6, 8], "min_samples_leaf": [1, 5, 10]}),
    "Random Forest": (RandomForestClassifier(random_state=42),
                      {"n_estimators": [100, 200], "max_depth": [4, 6, 8, None], "min_samples_leaf": [1, 3]}),
}
tuned, tuned_pred, tuned_proba, tuned_rows = {}, {}, {}, []
for name, (model, grid) in grids.items():
    search = GridSearchCV(model, grid, cv=skf, scoring="roc_auc")   # 5-fold CV on the training data only
    search.fit(Xc_train, yc_train)
    best = search.best_estimator_
    tuned[name] = best
    tuned_pred[name] = best.predict(Xc_test)
    tuned_proba[name] = best.predict_proba(Xc_test)[:, 1]
    tuned_rows.append({"Model": name, "Best_parameters": str(search.best_params_),
                       "CV_AUC": search.best_score_,
                       "Accuracy": accuracy_score(yc_test, tuned_pred[name]),
                       "F1": f1_score(yc_test, tuned_pred[name]),
                       "ROC_AUC": roc_auc_score(yc_test, tuned_proba[name])})
```

```
tuned_results = pd.DataFrame(tuned_rows).set_index("Model")
print(tuned_results[["Best_parameters"]].to_string())
print()
print(tuned_results.drop(columns="Best_parameters").sort_values("CV_AUC", ascending=False).round(3))
tuned_results.to_csv(f"{OUT}/classification_results_tuned.csv")
```

OUTPUT

```
                                                                Best_parameters
Model
Logistic Regression                                                  {'C': 0.1}
KNN                                                          {'n_neighbors': 21}
Decision Tree                              {'max_depth': 2, 'min_samples_leaf': 1}
Random Forest        {'max_depth': 4, 'min_samples_leaf': 3, 'n_estimators': 200}

                     CV_AUC  Accuracy     F1  ROC_AUC
Model
Logistic Regression   0.738     0.718  0.785    0.756
Random Forest         0.721     0.709  0.786    0.752
Decision Tree         0.713     0.709  0.758    0.712
KNN                   0.686     0.670  0.764    0.751
```

Tuned classifiers (sorted by cross-validated AUC).

Model	Best parameters	CV AUC	Test accuracy	Test F1	Test AUC
Logistic Regression	{'C': 0.1}	0.738	0.718	0.785	0.756
Random Forest	{'max_depth': 4, 'min_samples_leaf': 3, 'n_estimators': 200}	0.721	0.709	0.786	0.752
Decision Tree	{'max_depth': 2, 'min_samples_leaf': 1}	0.713	0.709	0.758	0.712
KNN	{'n_neighbors': 21}	0.686	0.670	0.764	0.751

Result: Tuning helped most where defaults were weak: Decision Tree AUC rose from 0.643 to 0.712 (a shallow tree of depth 2 generalises better) and KNN from 0.730 to 0.751 (k = 21 gives smoother predictions). Logistic Regression and Random Forest barely changed.

Step 6: Compare the models and pick the best one

The models are compared on the same test set. The best classifier is chosen by **cross-validated AUC**, not by the test score, so the test set stays an honest final check. The confusion matrix shows the types of mistakes, and the Random Forest shows which features matter most. The best models are saved with joblib.

CODE

```
best_clf = tuned_results["CV_AUC"].idxmax()            # chosen by cross-validation, not by the test set
comparison = pd.DataFrame({
    "Default_AUC": clf_results["ROC_AUC"], "Tuned_AUC": tuned_results["ROC_AUC"],
    "Default_Acc": clf_results["Accuracy"], "Tuned_Acc": tuned_results["Accuracy"]})
print(comparison.round(3))
print("\nBest classifier (highest cross-validated AUC):", best_clf)
print("Best regressor  (highest cross-validated R2) :", best_reg)
final = tuned[best_clf]
print("\nConfusion matrix of", best_clf, "(rows = actual, columns = predicted):")
print(pd.DataFrame(confusion_matrix(yc_test, tuned_pred[best_clf]),
                   index=["Rejected", "Approved"], columns=["Rejected", "Approved"]))
rf = tuned["Random Forest"]
importance = pd.Series(rf.feature_importances_, index=X_clf.columns).sort_values(ascending=False)
print("\nRandom Forest feature importance (top 6):")
print(importance.head(6).round(3))
plot_roc(tuned_proba, yc_test, tuned_results["ROC_AUC"])
plot_confusion(tuned_pred, yc_test)
plot_importance(importance)
joblib.dump(final, "models/best_classifier.joblib")
joblib.dump(reg_models[best_reg], "models/best_regressor.joblib")
print("\nSaved models/best_classifier.joblib and models/best_regressor.joblib")
```

OUTPUT

```
                     Default_AUC  Tuned_AUC  Default_Acc  Tuned_Acc
Model
Logistic Regression        0.758      0.756        0.728      0.718
KNN                        0.730      0.751        0.709      0.670
Decision Tree              0.643      0.712        0.660      0.709
Random Forest              0.748      0.752        0.738      0.709

Best classifier (highest cross-validated AUC): Logistic Regression
Best regressor  (highest cross-validated R2) : Linear Regression
```

```
Confusion matrix of Logistic Regression (rows = actual, columns = predicted):
          Rejected  Approved
Rejected        21        19
Approved        10        53

Random Forest feature importance (top 6):
Credit_Score        0.393
Loan_Amount         0.150
Annual_Income       0.138
Age                 0.107
Loan_Term_Months    0.045
Education           0.041
dtype: float64

Saved models/best_classifier.joblib and models/best_regressor.joblib
```

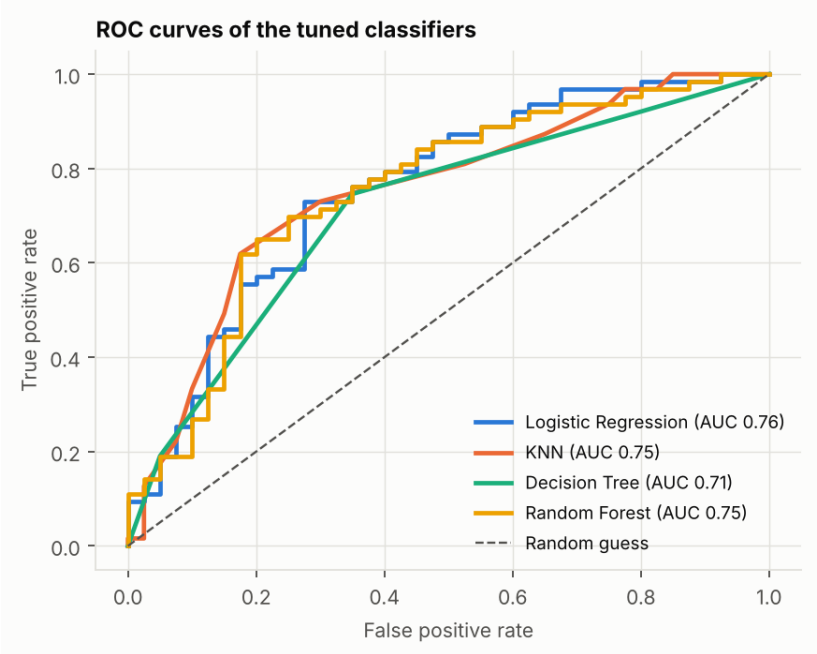


Figure 3 - ROC curves of the tuned classifiers: the closer to the top-left corner, the better. The dashed line is random guessing.

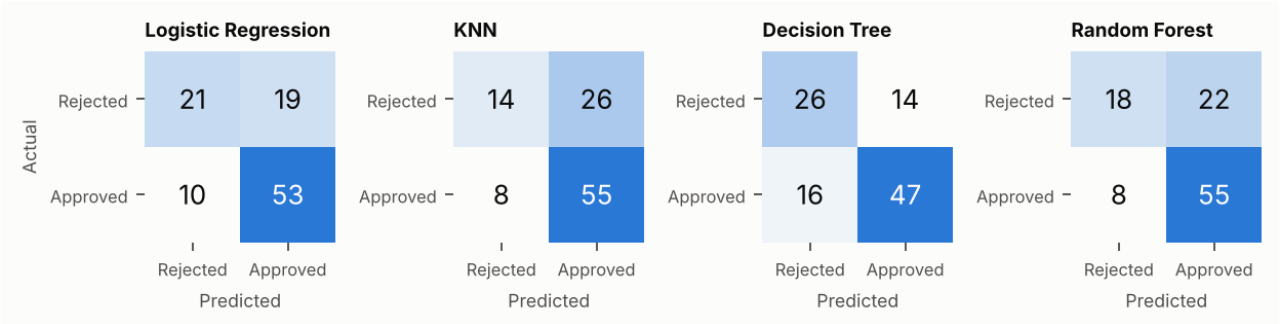


Figure 4 - Confusion matrices of the tuned classifiers on the test set (rows = actual, columns = predicted).

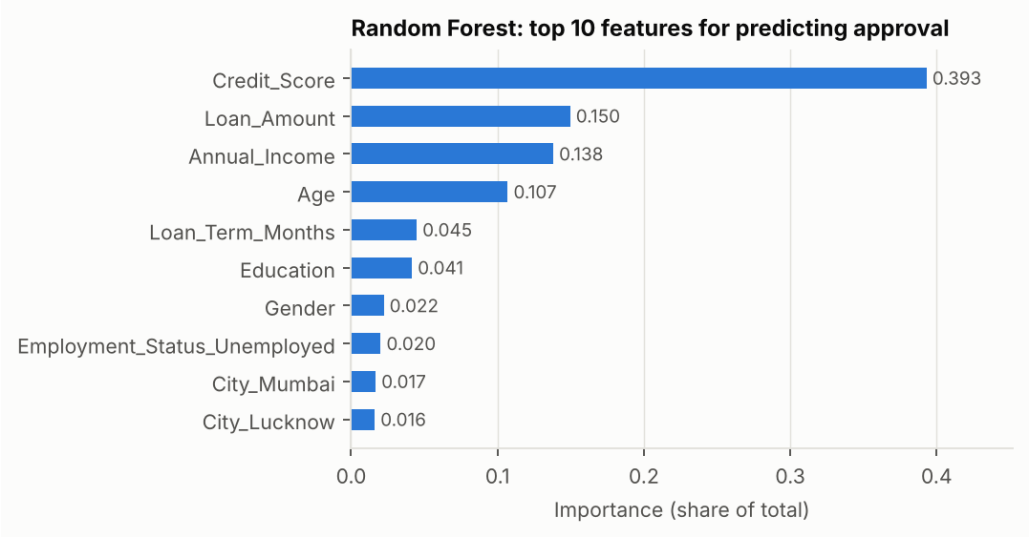


Figure 5 - The ten most important features in the Random Forest.

Result: Best classifier: Logistic Regression (CV AUC 0.738, test accuracy 0.718, F1 0.785, AUC 0.756). It correctly approved 53 and correctly rejected 21 applicants, but wrongly approved 19 rejected applicants and wrongly rejected 10 approved ones. **Best regressor: Linear Regression** (R2 0.760). Credit_Score is by far the most important feature for approval (0.39), followed by Loan_Amount and Annual_Income.

5. Final model comparison

Regression (predict annual income)

Model	MAE	RMSE	R2 (test)	R2 (cross-val.)	Average error in Rs.
Linear Regression	0.328	0.473	0.760	0.701	74,400
Random Forest	0.312	0.441	0.791	0.688	70,812
KNN Regressor	0.463	0.568	0.654	0.598	105,032
Decision Tree	0.377	0.504	0.727	0.547	85,509

Classification (predict loan approval), tuned models

Model	Best parameters	CV AUC	Test accuracy	Test F1	Test AUC
Logistic Regression	{'C': 0.1}	0.738	0.718	0.785	0.756
Random Forest	{'max_depth': 4, 'min_samples_leaf': 3, 'n_estimators': 200}	0.721	0.709	0.786	0.752
Decision Tree	{'max_depth': 2, 'min_samples_leaf': 1}	0.713	0.709	0.758	0.712
KNN	{'n_neighbors': 21}	0.686	0.670	0.764	0.751

How to read this: for both problems the simple linear models are as good as the more complex tree-based ones. When two models score almost the same, the simpler one is the better choice because it is faster, easier to explain and less likely to overfit.

6. Conclusion

- **Regression:** Linear Regression predicts annual income with R2 0.760 and an average error of about Rs. 74,400. Random Forest scored slightly higher on the test set (0.791) but slightly lower in cross-validation, so the difference is not meaningful.

- **Classification:** Logistic Regression is the best classifier by cross-validated AUC (0.738), with test accuracy 0.718 and AUC 0.756. Random Forest and KNN are very close.
- **Decision Tree** was the weakest model until it was tuned; a shallow tree (depth 2) generalised much better than a deep one.
- **Hyperparameter tuning** matters most for models that overfit or are sensitive to settings (Decision Tree, KNN).
- **Credit score** is the strongest signal for loan approval, which matches how lending works in practice.
- **Always compare several models** on the same split with several metrics, and choose with cross-validation rather than one test score.

7. Limitations and next steps

- The dataset is small (515 rows) and the test set has only 103 rows, so scores can move by several percentage points with a different split. Small gaps between models are not reliable.
- The data is synthetic and the approval label contains built-in randomness, so a score near 75% AUC is reasonable and perfect accuracy is not possible.
- In Week 1 the features were scaled on the whole dataset before splitting, which leaks a little information from the test rows. A cleaner approach is to put the scaler and the model in one Scikit-learn **Pipeline** so scaling is learned from the training data only.
- Next steps: use a Pipeline, try gradient boosting, handle class imbalance if it appears, and test on a real dataset.

Appendix: repository and how to run

```
week2-supervised-ml-models/
  README.md
  requirements.txt
  data/loan_applicants_clean.csv      cleaned data from Week 1
  src/train_models.py                 trains, tunes and evaluates all models
  src/build_report.py                 builds this PDF
  results/                            metric tables (CSV), charts, log
  models/                             saved best models (joblib)
  report/                             this PDF
```

To run: **pip install -r requirements.txt**, then **python3 src/train_models.py** from the repository root. The script prints every result and writes the tables, charts and saved models.